

第 40 讲

Initialization、Vanishing Gradient 与 Exploding Gradient

为什么参数起点和梯度尺度会决定深层网络能不能训起来

《面向 LLM / Agent Systems 科研的深度学习零基础课》

本讲定位

前面已经学会 Forward、Backward、Gradient、SGD、Momentum、Adam 与 AdamW。现在要补一个训练开始之前就已经发生的关键问题：**Parameter 应该从哪里出发？**如果所有 Weight 太小，信号和 Gradient 可能层层衰减；太大，则可能层层放大；如果所有隐藏神经元从完全相同的参数出发，它们甚至可能永远学成同一个东西。本讲把 Initialization、Vanishing/Exploding Gradient、Activation 饱和以及 Gradient Clipping 连成一条完整逻辑链。

学习目标

学完本讲，你应该能够：

1. 解释 Initialization 是什么、为什么训练开始前就必须认真处理；
2. 解释为什么“所有 Weight 都初始化成 0”会产生 symmetry problem；
3. 区分“Bias 可以为 0”和“隐藏层所有 Weight 都为 0”；
4. 理解为什么随机初始化不仅要随机，还必须控制数值尺度；
5. 看懂 fan-in、fan-out，以及 Xavier / Glorot 和 He / Kaiming 初始化的基本思想；
6. 从 Chain Rule 理解 Vanishing Gradient 与 Exploding Gradient；
7. 理解 Sigmoid/Tanh 饱和、ReLU、Residual、Normalization 与梯度传播之间的关系；
8. 解释 Gradient Clipping 做了什么、没有做什么；
9. 看懂 PyTorch 中常见的 `nn.init.xavier_normal_`、`kaiming_normal_`；
10. 在 LLM / Transformer / Agent Systems 实验中正确记录和判断 initialization、gradient norm、clipping 等训练设置。

1 Initialization 到底是什么？

神经网络训练的参数更新公式可以写成：

$$\theta_{t+1} = \theta_t - \eta g_t$$

这里 θ_0 就是训练开始前的参数初值。**Initialization（初始化）**解决的就是：

训练第 0 Step 时，Weight 和 Bias 应该取什么数？

它的输入不是训练样本, 而是模型结构信息, 例如某一层的输入维度、输出维度、Activation Function; 输出是一组初始 Parameter Tensor。

为什么不能说“反正后面会训练, 初始值随便就行”?

Backward 计算的 Gradient 本身就依赖当前 Parameter。初始参数会决定第一轮 Forward 的 Activation 尺度、第一轮 Loss、第一轮 Gradient, 进而影响后面整个训练轨迹。如果一开始就进入饱和区、数值爆炸区或完全对称状态, 优化器可能根本没有机会“慢慢修回来”。

初始化至少承担三个任务:

1. **打破对称性**: 不同隐藏神经元要有机会学习不同功能;
2. **控制 Forward 信号尺度**: Activation 不要层层趋近 0 或越来越大;
3. **控制 Backward 梯度尺度**: Gradient 不要在深层链路中快速消失或爆炸。

2 为什么隐藏层 Weight 不能全部初始化成 0?

考虑两个隐藏神经元, 它们接收相同输入 x , Activation 相同, 后续连接也完全对称。如果两个神经元的初始 Weight 和 Bias 完全一样, 那么它们的 Forward 输出一样; 由于它们在计算图中的角色也一样, Backward 得到的 Gradient 也一样。

本讲实验: 两个隐藏神经元全零初始化

输入:

$$x = [1, -2]$$

第一层两个神经元全部:

$$W_1 = 0, \quad b_1 = 0$$

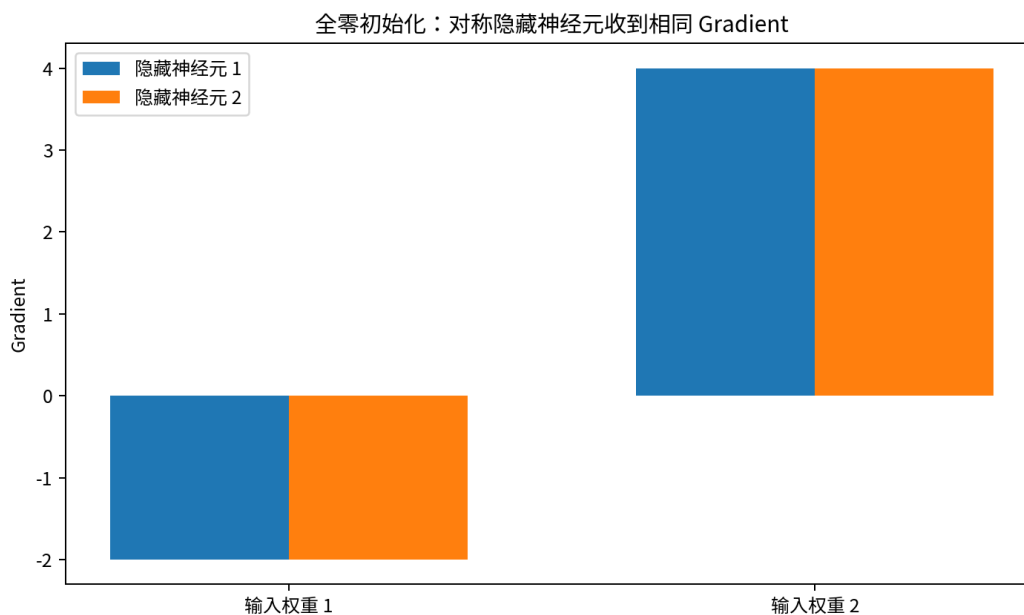
实验得到第一层 Gradient:

$$\frac{\partial L}{\partial W_1} = \begin{bmatrix} -2 & -2 \\ 4 & 4 \end{bmatrix}$$

两列完全相同。用相同 Learning Rate 更新以后:

$$W_1^{new} = \begin{bmatrix} 0.2 & 0.2 \\ -0.4 & -0.4 \end{bmatrix}$$

两个隐藏神经元仍然完全相同。



底层原因：Symmetry 没有被打破

如果两个隐藏单元从完全相同的状态出发，经历完全相同的计算，又收到完全相同的 Gradient，那么它们后续仍会同步移动。表面上网络有多个神经元，实际上多个神经元可能退化成“复制品”，浪费模型容量。

那 Bias 也不能初始化成 0 吗？

不是。常见做法中，**Bias 初始化为 0 通常没有问题**，因为 Weight 已经随机化，隐藏单元仍然能产生不同的 pre-activation，从而打破对称性。

常见误区

错误说法：“神经网络所有参数都不能初始化成 0。”

更准确的说法是：**对称隐藏单元的 Weight 不能全部以相同值开始，否则会出现 symmetry problem。**
Bias 是否为 0 要结合 Weight 和网络结构看。

3 随机初始化为什么还不够？关键是“尺度”

把 Weight 随机化可以打破对称性，但如果随机数过小：

$$|W| \ll 1$$

那么每层：

$$z = xW$$

可能不断把数值压小。

如果 Weight 过大：

$$|W| \gg 1$$

则同一个矩阵乘法可能不断放大数值。

所以真正的问题不是：

“随机还是不随机？”

而是：

“随机到多大的尺度？”

4 先建立 fan-in / fan-out 直觉

对于一个 Linear Layer：

$$y = xW + b$$

如果：

$$x \in \mathbb{R}^{D_{in}}, \quad y \in \mathbb{R}^{D_{out}}$$

那么通常称：

$$fan_in = D_{in}$$

$$fan_out = D_{out}$$

通俗地说：

- fan-in：一个输出神经元有多少输入连接进来；
- fan-out：一个输入神经元会向多少输出单元传播。

为什么初始化尺度要和 fan-in 有关？考虑：

$$z = \sum_{i=1}^n x_i w_i$$

如果每一项都是独立、均值近似 0，并令：

$$Var(x_i) = \sigma_x^2, \quad Var(w_i) = \sigma_w^2$$

粗略有：

$$Var(z) \approx n\sigma_x^2\sigma_w^2$$

其中：

$$n = fan_in$$

如果 σ_w 完全不随 n 改变，那么输入维度越大，输出方差越容易被放大。因此合理初始化通常让 Weight 的尺度随着 fan-in/fan-out 变化。

5 Xavier / Glorot Initialization: 尽量让信号尺度前后保持

Xavier 初始化常用于 Sigmoid/Tanh 或近似对称激活的经典场景。一个常见的 Xavier Normal 形式是：

$$Var(W) = \frac{2}{fan_in + fan_out}$$

因此标准差：

$$std(W) = \sqrt{\frac{2}{fan_in + fan_out}}$$

当：

$$fan_in \approx fan_out = n$$

时：

$$std(W) \approx \sqrt{\frac{1}{n}}$$

Xavier 的思想，不要只背公式

目标不是追求某个神奇常数，而是希望网络深度增加时，Forward Activation 和 Backward Gradient 的典型尺度不要系统性地层层变大或变小。

6 He / Kaiming Initialization: 为什么 ReLU 需要更大的尺度？

ReLU：

$$ReLU(z) = \max(0, z)$$

会把负半轴大量置 0。粗略地说，ReLU 会“丢掉”一部分信号能量，因此 He Initialization 常采用：

$$Var(W) \approx \frac{2}{fan_in}$$

也就是：

$$std(W) \approx \sqrt{\frac{2}{fan_in}}$$

这比简单的 $\sqrt{1/fan_in}$ 更大一些，用来补偿 ReLU 的截断效应。

本讲实验：30 层 ReLU 网络

宽度固定 256，仅比较初始化尺度，不训练。

过小初始化 $std = 0.01$ ：

$$Activation\ Std : 0.0937 \rightarrow 2.48 \times 10^{-10} \rightarrow 1.91 \times 10^{-29}$$

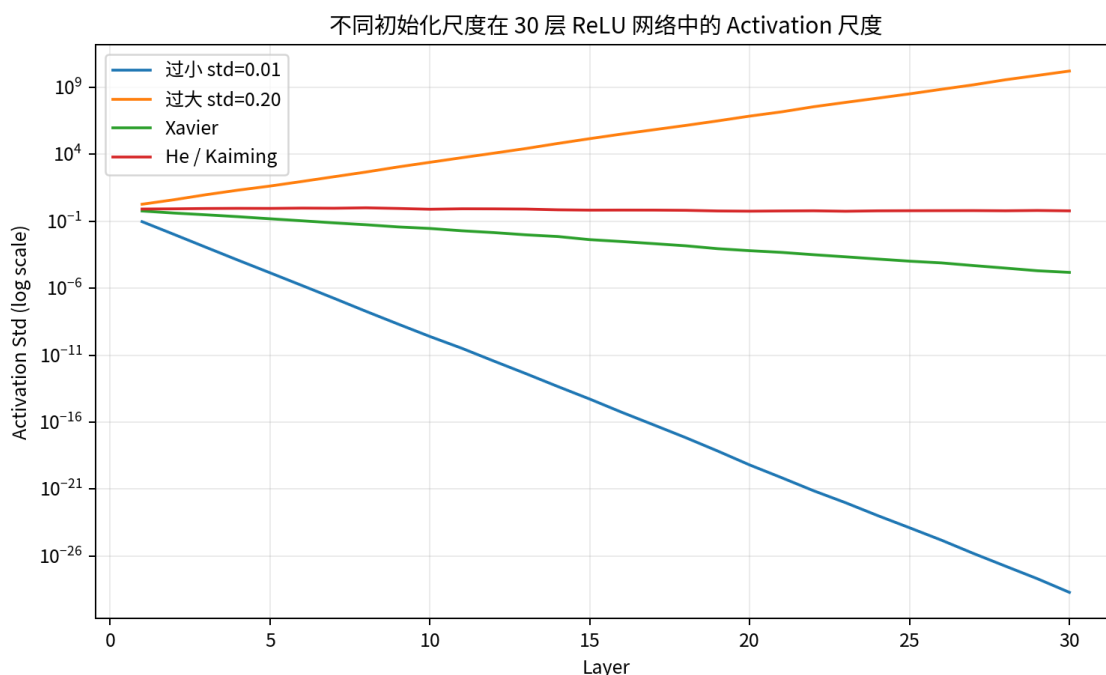
过大初始化 $std = 0.20$ ：

$$1.87 \rightarrow 2528.6 \rightarrow 1.66 \times 10^{10}$$

He Initialization：

$$0.823 \rightarrow 0.792 \rightarrow 0.599$$

在本实验中明显更稳定。



实验边界

这不是在证明“He 对所有网络永远最好”。本实验固定了 ReLU、网络宽度和简单随机矩阵结构，只是验证初始化尺度和 Activation 传播之间的机制。Transformer、GELU/SiLU、Residual、Normalization 下会有更多因素共同作用。

7 Vanishing Gradient: 梯度为什么会越传越小?

回到第 22 讲 Chain Rule。若：

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \cdots \rightarrow x_L \rightarrow L$$

则：

$$\frac{\partial L}{\partial x_0} = \frac{\partial L}{\partial x_L} \prod_{k=1}^L \frac{\partial x_k}{\partial x_{k-1}}$$

如果很多局部导数绝对值都小于 1，例如都约为 0.5：

$$0.5^{10} \approx 9.77 \times 10^{-4}$$

$$0.5^{20} \approx 9.54 \times 10^{-7}$$

$$0.5^{30} \approx 9.31 \times 10^{-10}$$

于是越靠前的层收到的 Gradient 越小，几乎无法更新，这就是 **Vanishing Gradient** 的最小数学模型。

8 Exploding Gradient: 同一条 Chain Rule 也可能反过来

如果很多局部导数绝对值都大于 1，例如 1.5：

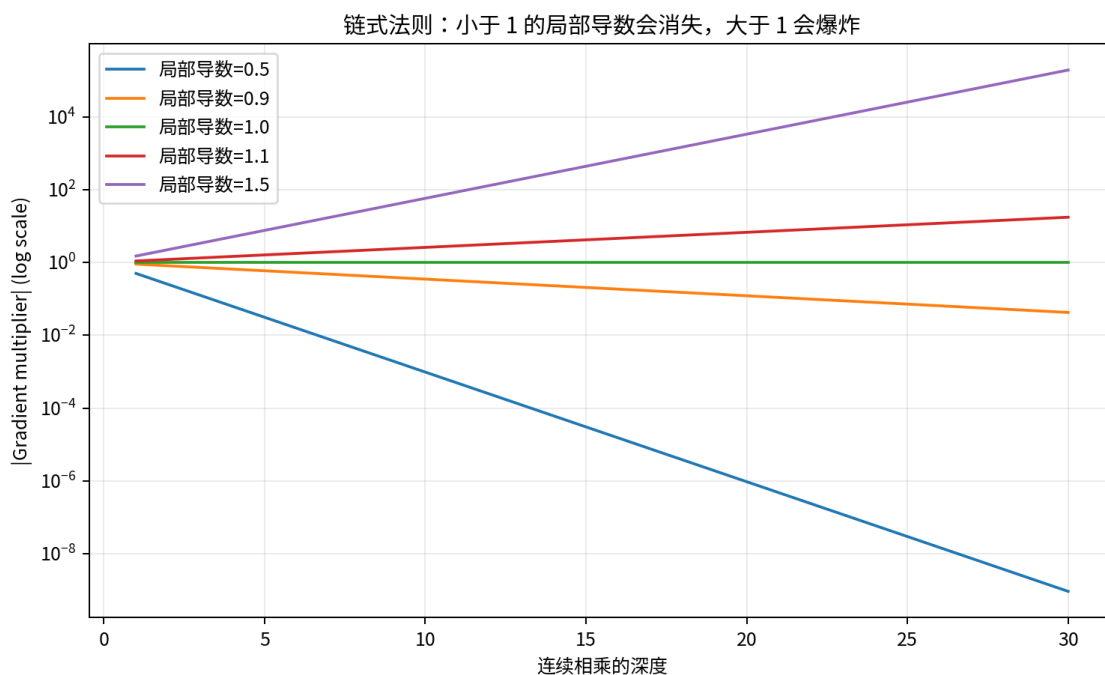
$$1.5^{10} \approx 57.7$$

$$1.5^{20} \approx 3325$$

$$1.5^{30} \approx 1.92 \times 10^5$$

Gradient 会被不断放大，导致：

- Parameter Update 极大；
- Loss 剧烈震荡或直接发散；
- 数值可能出现 inf / nan；
- Optimizer state 也可能被异常 Gradient 污染。



Vanishing / Exploding 的本质

不是“网络深了自动就坏”，而是很多 **Jacobian** / 局部导数连续相乘以后，整体尺度发生系统性衰减或放大。初始化、Activation、网络结构、Normalization、Residual Connection 都会改变这个乘积。

9 Activation Function 为什么会影响 Gradient?

Tanh 的饱和区

Tanh:

$$a = \tanh(z)$$

导数：

$$\frac{da}{dz} = 1 - \tanh^2(z)$$

当 $|z|$ 很大时：

$$\tanh(z) \approx \pm 1$$

所以：

$$\frac{da}{dz} \approx 0$$

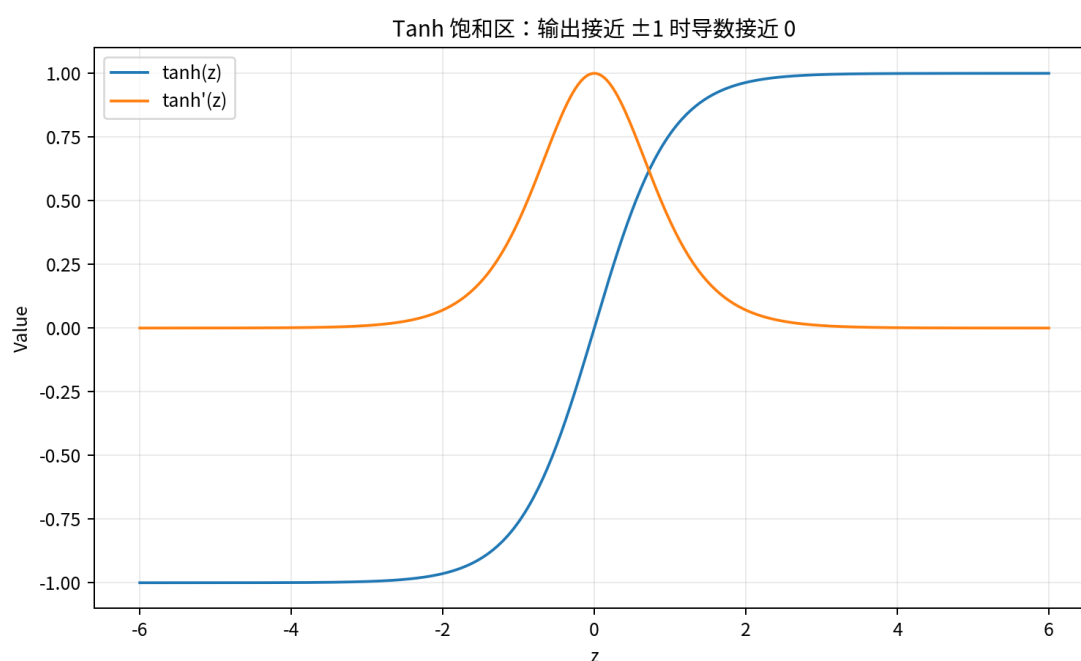
本讲实验：

$$z = \pm 5$$

时导数只有约：

$$1.82 \times 10^{-4}$$

而 $z = 0$ 时导数为 1。



Sigmoid 也有相似饱和问题

Sigmoid：

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

最大导数只有 0.25，在两端饱和区又接近 0。因此非常深的传统网络如果大量依赖 Sigmoid，Backward 更容易衰减。

ReLU 解决了什么，又带来什么？

对于 $z > 0$ ：

$$\text{ReLU}'(z) = 1$$

因此正区间不会像 Sigmoid/Tanh 那样自动压小 Gradient。但 $z < 0$ 时：

$$\text{ReLU}'(z) = 0$$

所以某些神经元可能长期处于关闭区，产生 dead ReLU 问题。

GELU / SiLU 更平滑，但这并不意味着“自动彻底解决所有 Gradient 问题”。

10 Forward 爆炸与 Gradient 爆炸不是同一个词，但经常相关

需要区分：

- **Activation explosion**：Forward 中间值越来越大；
- **Gradient explosion**：Backward Gradient 越来越大；
- **Activation collapse**：Forward 信号越来越接近 0；
- **Gradient vanishing**：Backward Gradient 越来越接近 0。

它们可能由同样的 Weight Scale、Activation、深度等原因共同造成，但不是严格同一个量。科研实验中不能只看 Loss，就声称“发生了梯度爆炸”；最好真正记录：

$$\|g\|, \quad \text{Activation Mean/Std}, \quad \text{Parameter Norm}$$

等证据。

11 Gradient Clipping：如果 Gradient 已经太大怎么办？

一种常见工程措施是 **Gradient Clipping**。最常见的是 Global Norm Clipping。

把所有参数 Gradient 看成一个长向量 g ，计算：

$$\|g\|_2$$

若：

$$\|g\|_2 > C$$

则整体缩放：

$$g \leftarrow g \frac{C}{\|g\|_2}$$

于是方向基本保持，但总长度被限制在阈值 C 。

本讲实验中：

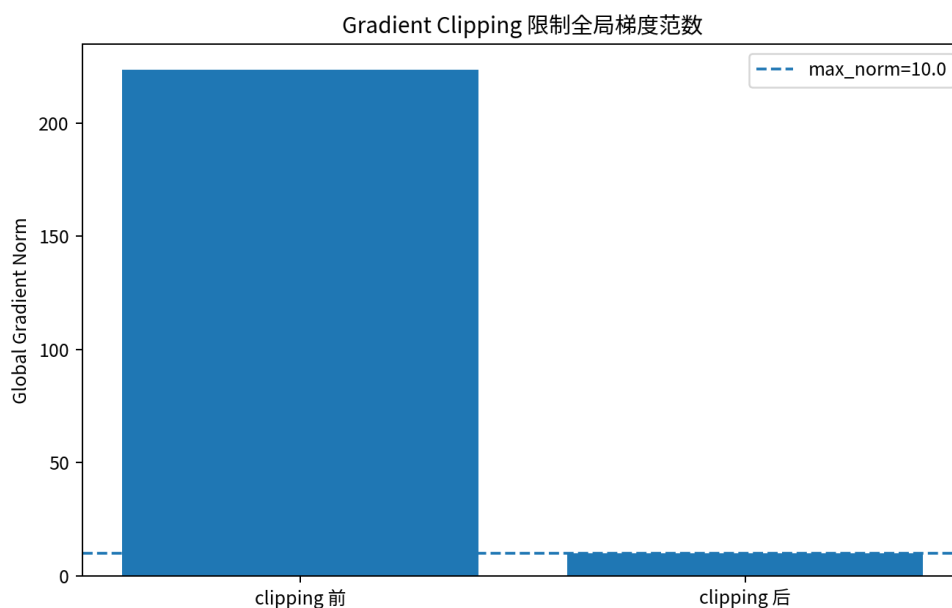
$$\|g\| = 223.6068$$

设置：

$$\text{max_norm} = 10$$

clipping 后:

$$\|g\| \approx 10.0000$$



PyTorch:

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

通常放在:

```
loss.backward()  
clip_grad_norm_(...)  
optimizer.step()
```

之间。

Gradient Clipping 不是根治

Clipping 可以阻止一次异常 Gradient 把 Parameter 推得太远, 但它并不会自动修复错误初始化、错误 LR、数值不稳定、数据异常或模型结构问题。如果每一步都严重 clipping, 应该继续追查根因。

12 PyTorch 中怎样做 Xavier / Kaiming Initialization?

常见写法:

```
layer = nn.Linear(512, 512)  
nn.init.xavier_normal_(layer.weight)  
nn.init.zeros_(layer.bias)
```

或者 ReLU 网络:

```
nn.init.kaiming_normal_(layer.weight, nonlinearity="relu")  
nn.init.zeros_(layer.bias)
```

本讲实际检查一个 512×512 Weight:

$$Xavier\ std \approx 0.04415$$

理论量级:

$$\sqrt{1/512} \approx 0.04419$$

Kaiming:

$$std \approx 0.06264$$

理论量级:

$$\sqrt{2/512} = 0.0625$$

13 初始化到底是“训练前一次”还是训练中一直发生?

普通情况下, Initialization 发生在模型建立时:

$$\theta_0 \leftarrow Init()$$

之后参数由 Optimizer 更新, 不会每个 Epoch 再重新初始化。

非常危险的错误

如果你在训练循环中不小心重新创建 Model, 或者每个 Epoch 重新调用初始化函数, 相当于不断把学到的 Parameter 擦掉。现象通常是 Loss 每轮又回到类似起点。

随机初始化还涉及 **Random Seed**。严格复现实验时, 要记录:

- 初始化方法;
- seed;
- PyTorch / CUDA 版本;
- 是否使用 deterministic setting;
- 数据 shuffle seed。

14 Transformer / LLM 中这些知识在哪里?

你以前在 LLM 课程里见过:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

以及 FFN:

$$XW_1 \rightarrow GELU/SiLU \rightarrow W_2$$

这些 W_Q, W_K, W_V, W_1, W_2 在训练开始前都需要初始化。

现代 Transformer 之所以能堆很多层，不是因为“Gradient 消失问题已经不存在”，而是同时依赖多种机制改善信号传播，例如：

- 合理初始化尺度；
- Residual Connection；
- LayerNorm / RMSNorm；
- GELU / SiLU 等 Activation；
- 合理 Learning Rate 与 Warmup；
- 某些架构还会对 Residual Branch 使用特殊 scaling。

后面第 47 讲学习 LayerNorm / RMSNorm、第 56 讲重新组装 Transformer Block 时，这些知识会重新汇合。

与 Agent Systems 科研的关系

很多 Agent Systems 研究主要调用预训练 LLM API，并不自己训练基础模型，所以你不需要成为 Initialization 专家。但如果进行 LoRA/QLoRA、Reward Model、Policy Fine-tuning、Embedding Model 微调或小型本地模型实验，Initialization、Gradient Norm、Clipping 和数值稳定性就可能直接决定实验是否可用。

更重要的是：看到训练失败时，不要把“Loss 不下降”简单归因于模型能力。它可能只是 optimization failure。

15 怎样诊断：初始化 / 梯度传播是不是出了问题？

可以建立一个最低排查顺序：

1. 看 Training Loss 是否一开始就是 nan/inf；
2. 记录每层 Activation Mean / Std；
3. 记录 Global Gradient Norm；
4. 查看是否某些层长期 Gradient 近似 0；
5. 查看是否 Gradient Norm 突然增长几个数量级；
6. 检查 LR 是否过大；
7. 检查 Weight Initialization 是否和 Activation 匹配；
8. 检查输入数据是否已经存在极端值或错误尺度；
9. 必要时尝试 clipping，但同时继续找根因。

科研中不要只说“训练不稳定”

更有证据的写法是：

- “第 300 Step 起 global gradient norm 从约 2 上升到 10^4 ，随后 Loss 出现 NaN”；

- “前 20 层 activation std 从 0.8 逐层衰减到 10^{-5} ”;
- “加入 clipping 后 NaN 消失, 但 gradient norm 每一步都触顶, 说明 clipping 可能只是抑制症状”。这比笼统地说“模型梯度爆炸”更符合科研证据链。

16 再看一个具体数字: fan-in 为什么会让方差放大?

假设一个神经元有 1000 个输入:

$$z = \sum_{i=1}^{1000} x_i w_i$$

为了建立直觉, 暂时假设:

$$\text{Var}(x_i) = 1$$

且每个输入与 Weight 近似独立、均值为 0。

如果 Weight 也取:

$$\text{Var}(w_i) = 1$$

那么粗略有:

$$\text{Var}(z) \approx 1000$$

标准差约为:

$$\text{std}(z) \approx \sqrt{1000} \approx 31.6$$

也就是说, 输入明明只有标准差 1, 经过一层 Linear 后数值尺度就可能被放大到几十。

如果改成:

$$\text{Var}(w_i) = \frac{1}{1000}$$

则:

$$\text{Var}(z) \approx 1000 \times 1 \times \frac{1}{1000} = 1$$

这就是为什么初始化方差通常要随着 fan-in 增大而减小。

不要把初始化公式理解成魔法常数

$1/\text{fan_in}$ 、 $2/\text{fan_in}$ 或 $2/(\text{fan_in} + \text{fan_out})$ 的共同思想, 都是让“很多输入项相加”以后, 总体数值尺度不要因为维度本身而机械变得越来越大。不同 Activation 对信号有不同截断/缩放, 因此会对应不同的常数因子。

17 为什么 Initialization 同时影响 Backward? 看 Linear 的反传公式

Linear Forward:

$$Y = XW + b$$

上一讲已经知道, Backward 有:

$$dX = dY W^T$$

$$dW = X^T dY$$

这里可以直接看到：传回上一层的 **Gradient** dX 会乘上当前 **Weight** W 。

如果很多层的 W 都具有非常小的典型尺度，那么 Backward 中的 dX 可能层层缩小；如果很多层的 W 都过大，Backward 也可能层层放大。

所以 Initialization 并不是只控制 Forward：

W 同时出现在 Forward 信号传播和 Backward 梯度传播中

这也是 Xavier / He 初始化为什么会同时讨论 “forward variance” 和 “backward variance” 的原因。

一个实用的 Shape + Scale 心智模型

看一个 Linear Layer 时，可以同时问四件事：

1. X 的 Shape 是什么？
2. W 的 Shape 是什么？
3. XW 的数值尺度可能怎样变化？
4. dYW^T 的 Gradient 尺度可能怎样变化？

Shape 决定 “能不能乘”，Scale 决定 “乘完以后会不会数值失控”。

18 Residual Connection 为什么有助于深层梯度传播？

Transformer 的核心结构之一是 Residual Connection。最简单写成：

$$y = x + F(x)$$

对 x 求导：

$$\frac{\partial y}{\partial x} = I + \frac{\partial F}{\partial x}$$

Backward：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \left(I + \frac{\partial F}{\partial x} \right)$$

直觉上，除了经过复杂 $F(x)$ 的路径，还存在一条接近 identity 的直接路径。这让 Gradient 不必完全依赖很多层 “小导数连乘” 才能传回去。

Residual 也不是万能药

Residual Connection 能改善梯度流，但并不意味着初始化和 Normalization 不重要。深层 Transformer 仍需要整体 recipe：初始化、Residual scaling、Norm placement、LR、Warmup、数值精度等共同配合。

19 Normalization 和 Initialization 是什么关系？

后面会系统学习 BatchNorm、LayerNorm、RMSNorm。这里只建立最低直觉：Normalization 会对中间 Activation 的尺度进行重新规范化，从而降低不同层数值尺度漂移的风险。

但两者职责不同：

- Initialization：决定训练第 0 Step 的 Parameter 起点；
- Normalization：在每次 Forward 中动态调整某些中间数值尺度；
- Residual：提供信息和梯度的快捷路径；
- Optimizer / LR：决定参数随后怎样移动。

所以看到一个现代 Transformer 很稳定，不能只归功于某一个组件。系统稳定性往往来自多个机制的组合。

20 一个最小的 PyTorch 自定义初始化模板

科研代码里常看到模型建立以后再统一初始化：

```
def init_weights(module):
    if isinstance(module, nn.Linear):
        nn.init.kaiming_normal_(module.weight, nonlinearity="relu")
        if module.bias is not None:
            nn.init.zeros_(module.bias)

model.apply(init_weights)
```

这段代码做了四件事：

1. 遍历 Module tree；
2. 只处理 Linear；
3. Weight 使用 Kaiming；
4. Bias 置 0。

不要复制初始化代码却不看 Activation

如果模型实际用 GELU/SiLU、特殊 residual scaling，或者作者已经在 Module 内部定义了自己的 initialization，你再无脑调用一遍 `model.apply(init_weights)`，可能把原本设计好的初始化覆盖掉。科研复现时要先读模型源码。

21 训练日志里怎样区分“初始化坏”与“后期优化坏”？

Initialization 问题常在训练最开始就暴露：

- Step 0/1 Activation 已经极小或极大；

- 初始 Loss 就异常大、NaN 或 Inf;
- 某些层从第一步起 Gradient 就几乎为 0;
- 不同 seed 下是否都出现相同失败模式。

后期优化问题则可能表现为:

- 开始训练正常, 某个 Step 后 Gradient Norm 突然上升;
- LR Scheduler 切换以后开始震荡;
- 数据分布变化或某批异常数据之后出现 NaN;
- Optimizer state 长期积累后出现异常。

这不是绝对诊断规则, 但能帮助你建立“现象发生在什么时候”的证据链, 而不是看到 NaN 就说一句“梯度爆炸”。

22 常见误区集中澄清

误区 1: 所有参数都随机初始化最好

Bias 常常可以为 0; Normalization 的 scale / bias 也常有特定初始化。初始化是按参数角色设计, 不是“全部随机”。

误区 2: Xavier / He 能保证训练一定稳定

不能。它们只是改善初始信号尺度。Optimizer、LR、Normalization、Residual、数据、数值精度等仍然影响训练。

误区 3: Gradient 很小就是模型收敛

不一定。前面已经见过 plateau、Activation saturation、dead ReLU, 都可能导致 Gradient 很小但 Loss 并不好。

误区 4: Clipping 后训练不炸, 就说明问题解决

不一定。如果根因是 LR 极大或初始化错误, clipping 可能长期截断所有更新, 训练虽然不 NaN, 却也可能学得很差。

23 本讲知识链总装

现在把本讲压缩成一条链:

Initialization → *Forward Scale* → *Local Derivative*
→ *Chain Rule* → *Gradient Scale* → *Optimizer Update*

随机初始化先打破 symmetry; Xavier / He 尝试让信号尺度在网络深度中更健康; 若局部导数长期小于 1, Gradient 可能消失; 若长期大于 1, Gradient 可能爆炸; Gradient Clipping 可以限制异常更新, 但

不能替代正确的初始化与优化设置。

本讲最低掌握标准

如果以后看到训练代码：

```
nn.init.kaiming_normal_(layer.weight)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
```

你应该能解释：第一行控制训练起点的 Weight Scale；第二行通过 Chain Rule 得到 Gradient；第三行在更新前限制全局梯度范数；第四行才真正修改 Parameter。

下一讲预告

第 41 讲进入新的阶段：**Overfitting 与 Generalization**。我们会第一次把“Training Loss 降下来”与“模型真的学会可泛化规律”严格分开，并通过 Train / Validation 曲线观察过拟合是怎样发生的。